

Algorithmen für den 2. Test

Minimales Vertex Cover

i NP Spezialfälle

```

Vertex-Cover( $G, k$ ):
if  $G$  enthält keine Kanten
    return true
if  $G$  enthält  $> k(n - 1)$  Kanten
    return false
Sei  $(u, v)$  eine beliebige Kante von  $G$ 
 $a \leftarrow \text{Vertex-Cover}(G - \{u\}, k - 1)$ 
 $b \leftarrow \text{Vertex-Cover}(G - \{v\}, k - 1)$ 
return  $a$  OR  $b$ 
  
```

Laufzeit: $O(2^k kn)$

i Branch-and-Bound Algorithmus

```

MinVertexCover-BranchAndBound( $G, C$ ):
 $U \leftarrow$  gültige heuristische Lösung für  $(G, C)$  mit Greedyheuristik
 $L \leftarrow$  untere Schranke für  $(G, C)$  mit Matchingheuristik
 $\Pi \leftarrow \{((G, C), L, U)\}$ 
while  $\exists I' \in \Pi$ 
    Entferne  $I' = ((G', C'), L', U')$  aus  $\Pi$ 
    if  $L' < U$ 
         $u_{\max} \leftarrow$  Knoten mit maximalem Grad in  $G'$ 
        Erzeuge Teilinstanzen  $I_1 = (G' - \{u_{\max}\}, C \cup \{u_{\max}\})$  und
             $I_2 = (G' - \{u_{\max}\} - N(u_{\max}), C \cup N(u_{\max}))$ 
        Berechne für  $I_1, I_2$  gültige heuristische Lösungen mit Greedyheuristik  $\rightarrow$  obere Schranken  $U_1, U_2$ 
        Berechne für  $I_1, I_2$  lokale untere Schranken  $L_1, L_2$  mit Matchingheuristik
         $\Pi \leftarrow \Pi \cup \{(I_1, L_1, U_1), (I_2, L_2, U_2)\}$ 
        if  $\min\{U_1, U_2\} < U$ 
             $U \leftarrow \min\{U_1, U_2\}$ 
return beste gefundene Lösung mit Wert  $U$ 
  
```

② 2-Approximationsalgorithmus

Approx-Vertex-Cover(G):

$C \leftarrow \emptyset$

while $E \neq \emptyset$

Wähle eine beliebige Kante $(u, v) \in E$

$C \leftarrow C \cup \{u, v\}$

Entferne aus E alle Kanten, die inzident zu u oder v sind

return C

Laufzeit: $O(n + m)$

$n = |V|$

$m = |E|$

② 2-Approximationsalgorithmus (Alternative)

Approx-Vertex-Cover2(G):

$C \leftarrow \emptyset$

while $E \neq \emptyset$

Wähle einen Knoten u mit maximalem Grad im aktuellen Graphen

$C \leftarrow C \cup \{u\}$

Entferne aus E alle Kanten, die inzident zu u sind

return C

② Lokale Suche

Nachbarschaftsstruktur: $C' \in N(C)$ wenn C' aus C durch Löschen eines einzigen Knotens erzeugt werden kann und noch immer ein Vertex Cover ist. $N(C) = O(|V|)$.

Lokale Suche: Starte mit Vertex Cover C , z.B. $C = V$.

Wenn es ein $C' \in N(C)$ gibt, das ein gültiges Vertex Cover ist, so ist dieses immer eine bessere Lösung, da $|C'| = |C| - 1$.

Ersetze C durch ein solches C' .

Laufzeit: $O(n)$

$n = |V|$

Maximales Independent Set

NP Spezialfälle auf Bäumen

Independent-Set-In-A-Forest(T):

$S \leftarrow \emptyset$

while T hat zumindest eine Kante

Sei $e = (u, v)$ eine Kante, sodass v ein Blatt ist

Füge v zu S hinzu

Lösche aus V die Knoten u und v (und alle zu diesen Knoten inzidenten Kanten)

$S \leftarrow S \cup V$

return S

Laufzeit: $O(n)$ durch Postorder Durchmusterung

NP Spezialfälle auf gewichteten Bäume

Weighted-Independent-Set-In-A-Tree(T):

Wähle eine Wurzel r aus

foreach Knoten u von T in Postorder

if u ist ein Blatt

$M_{\text{in}}[u] \leftarrow w_u$

$M_{\text{out}}[u] \leftarrow 0$

else

$M_{\text{in}}[u] \leftarrow w_u + \sum_{v \in \text{Nachfolger}(u)} M_{\text{out}}[v]$

$M_{\text{out}}[u] \leftarrow \sum_{v \in \text{Nachfolger}(u)} \max\{M_{\text{in}}[v], M_{\text{out}}[v]\}$

return $\max\{M_{\text{in}}[r], M_{\text{out}}[r]\}$

Laufzeit: $O(n)$ durch Postorder Durchmusterung

Coloring Problem

i NP Spezialfälle auf Intervallgraphen

```

Interval-Coloring( $\mathcal{I}$ ):
Sortiere Intervallgrenzen aufsteigend in Liste  $\mathcal{L}$ 
 $\text{col}_{\max} \leftarrow 0$ 
Initialisiere leere Queue  $Q$ 
foreach Punkt  $a$  von  $\mathcal{L}$ 
    if  $a$  ist ein Startpunkt
        if  $Q$  ist leer
            färbe  $I = [a, b]$  mit Farbe  $\text{col}_{\max}$ 
             $\text{col}_{\max} \leftarrow \text{col}_{\max} + 1$ 
        else
            entferne  $c = Q.\text{head}$  und färbe  $I = [a, b]$  mit Farbe  $c$ 
    else //  $a$  ist ein Endpunkt
        füge Farbe von  $I = [b, a]$  in  $Q$  ein
return Färbung von  $\mathcal{I}$  und Anzahl  $\text{col}_{\max}$ 

```

Laufzeit: $O(n \log n)$

Rucksackproblem

① Branch-and-Bound Algorithmus (Verbesserte Version)

```

Enum( $z, w_{\text{curr}}, g_{\text{curr}}, \vec{x}$ ):
  if  $g_{\text{curr}} \leq G$ 
    if  $w_{\text{curr}} > w_{\text{max}}$ 
       $w_{\text{max}} \leftarrow w_{\text{curr}}$ 
       $\vec{x}_{\text{best}} \leftarrow \vec{x}$ 
    for  $i \leftarrow z + 1$  bis  $n$ 
       $U' \leftarrow w_{\text{curr}} + (G - g_{\text{curr}}) \cdot \frac{w_i}{g_i}$ 
      if  $U' > w_{\text{max}}$ 
         $x_i \leftarrow 1$ 
        Enum( $i, w_{\text{curr}} + w_i, g_{\text{curr}} + g_i, \vec{x}$ )
         $x_i \leftarrow 0$ 

```

(Werte werden anfangs absteigend nach $\frac{w_i}{g_i}$ sortiert)

① Dynamische Programmierung (Bottom-up)

```

for  $g \leftarrow 0$  bis  $G$ 
   $M[0, g] \leftarrow 0$ 

for  $i \leftarrow 1$  bis  $n$ 
  for  $g \leftarrow 0$  bis  $G$ 
    if  $g_i > g$ 
       $M[i, g] \leftarrow M[i - 1, g]$ 
    else
       $M[i, g] \leftarrow \max\{M[i - 1, g], w_i + M[i - 1, g - g_i]\}$ 
return  $M[n, G]$ 

```

Laufzeit und Platzbedarf: $O(nG)$

G = Rucksackkapazität

n = Anzahl an Elemente

Fibonacci

① Dynamische Programmierung (Rekursiv)

```
for  $i \leftarrow 1$  bis  $n$ 
   $F[i] \leftarrow \text{leer}$ 

Fibonacci( $n$ ):
if  $F[n]$  ist leer
  if  $n = 1$  oder  $n = 2$ 
     $F[n] \leftarrow 1$ 
  else
     $F[n] \leftarrow \text{Fibonacci}(n - 1) + \text{Fibonacci}(n - 2)$ 
return  $F[n]$ 
```

Laufzeit: $O(n)$

① Dynamische Programmierung (Iterativ)

```
Linear-Fibonacci( $n$ ):
 $F[1] \leftarrow 1$ 
 $F[2] \leftarrow 1$ 
for  $i \leftarrow 3$  bis  $n$ 
   $F[i] \leftarrow F[i - 1] + F[i - 2]$ 
return  $F[n]$ 
```

Laufzeit: $O(n)$

Interval Scheduling

Dynamische Programmierung

```
for  $j \leftarrow 1$  bis  $n$ 
   $M[j] \leftarrow \text{leer}$ 
 $M[0] \leftarrow 0$ 

M-Compute-Opt( $j$ ):
  if  $M[j]$  ist leer
     $M[j] \leftarrow \max(w_j + \text{M-Compute-Opt}(p(j)), \text{M-Compute-Opt}(j - 1))$ 
  return  $M[j]$ 
```

(Jobs sortiert nach Beendigungszeit)

Laufzeit von M-Compute-Opt(n): $O(n)$

Insgesamte Laufzeit: $O(n \log n)$

① Dynamische Programmierung (Bottom-up)

```

Iterative-Compute-Opt():
M[0] ← 0
for  $j \leftarrow 1$  bis  $n$ 
     $M[j] \leftarrow \max(w_j + M[p(j)], M[j - 1])$ 

```

(Jobs sortiert nach Beendigungszeit)

Laufzeit von Iterative-Computer-Opt(): $O(n)$

Laufzeit: $O(n \log n)$

① Finden einer Lösung nach ausführen eines dynamischen Programmierungsalgorithmus

```

Find-Solution( $j$ ):
if  $j = 0$ 
    Keine Ausgabe
elseif  $w_j + M[p(j)] > M[j - 1]$ 
    Gib  $j$  aus
    Find-Solution( $p(j)$ )
else
    Find-Solution( $j - 1$ )

```

Laufzeit: $O(n)$

Segmented Least Squares:

① Dynamische Programmierung

```

Segmented-Least-Squares(  $P = \{p_1, p_2, \dots, p_n\}$  )
   $M[0] = 0$ 
  for  $j \leftarrow 1$  bis  $n$ 
    for  $i \leftarrow 1$  bis  $j$ 
      berechne Fehler  $e(i, j)$  für Punkte  $p_i, \dots, p_j$ 

  for  $j \leftarrow 1$  to  $n$ 
     $M[j] = \min_{1 \leq i \leq j} (M[i-1] + e(i, j) + c)$ 

  return  $M[n]$ 

```

Laufzeit: $O(n^3)$

Shortest Path

① Dynamische Programmierung

```

Push-Based-Shortest-Path-DP( $G, s, t$ ):
  foreach Knoten  $v \in V$ 
     $M[v] \leftarrow \infty$ 
    Nachfolger[ $v$ ]  $\leftarrow \emptyset$ 

   $M[t] = 0$ 
  for  $i \leftarrow 1$  bis  $n - 1$ 
    foreach Kante  $(v, w) \in E$ 
      if  $M[v] > M[w] + c_{vw}$ 
         $M[v] \leftarrow M[w] + c_{vw}$ 
        Nachfolger[ $v$ ]  $\leftarrow w$ 
    if kein  $M[w]$  Wert ändert sich in Iteration  $i$ , stop.
  return  $M[s]$ 

```

Laufzeit: $O(mn)$, Platzbedarf: $O(m + n)$

$m = |E|$

$n = |V|$

Traveling Salesman Problem

Spanning-Tree-Heuristik für das sym. TSP

Spanning-Tree-Heuristik: Tour wird aus einem Minimum Spanning Tree (MST) für den gegebenen Graphen G abgeleitet:

Vorgehen:

1. Bestimme einen MST (V, B_1) von G .
2. Verdopple alle Kanten in B_1 , das ergibt Graph (V, B_2) .
3. Bestimme eine **Eulertour** F im Graphen (V, B_2) . Gib dieser Tour eine Orientierung, wähle einen Knoten $i \in V$, markiere i , setze $p \leftarrow i, T \leftarrow \emptyset$.
4. Sind alle Knoten markiert, setze $T \leftarrow T \cup \{(p, i)\}$ und retourniere T als Ergebnis-Tour.
5. Laufe von p entlang der Orientierung von F bis ein unmarkierter Knoten q erreicht ist. Setze $T \leftarrow T \cup \{(p, q)\}$, markiere q , setze $p \leftarrow q$ und gehe zu (4).

Laufzeit: $O(n^2)$

$n = |V|$

r-opt lokale Suche

mit First-Improvement Schrittfunktion

Vorgehen:

- (1) Sei $E(T) = \{(i_1, i_2), (i_2, i_3), \dots, (i_n, i_1)\}$ die Menge der Kanten der aktuellen Tour T und sei $i_{n+1} = i_1$.
- (2) Sei $Z = \{\{(i_p, i_{p+1}), (i_q, i_{q+1})\} \subset T \mid 1 \leq p, q \leq n \wedge p+1 < q\}$
sei die Menge aller Paare nicht nebeneinanderliegender Kanten
- (3) Für alle Kantenpaare $\{(i_p, i_{p+1}), (i_q, i_{q+1})\}$ aus Z :
Falls $c_{i_p i_{p+1}} + c_{i_q i_{q+1}} > c_{i_p i_q} + c_{i_{p+1} i_{q+1}}$:
 - $T = T \setminus \{(i_p, i_{p+1}), (i_q, i_{q+1})\} \cup \{(i_p, i_q), (i_{p+1}, i_{q+1})\}$
 - gehe zu (2)
- (4) retourniere T .

Laufzeit: $O(n^r)$ (unwahrscheinlicher worst-case: $O(n!)$)

$n = |V|$

① Evolutionärer Algorithmus

```

Edge-Recombination( $T^1, T^2$ ):
Beginne bei einem beliebigen Startknoten  $v \leftarrow v_0$ ,  $T \leftarrow \{\}$ 
while es existieren noch unbesuchte Knoten
    Sei  $W$  Menge noch unbesuchten Knoten, welche in
     $T^1 \cup T^2$  adjazent zu  $v$  sind
    if  $W \neq \{\}$ 
        Wähle einen Nachfolgeknoten  $w \in W$  zufällig aus
    else
        Wähle einen zufälligen noch nicht
        besuchten Nachfolgeknoten  $w$ 
     $T \leftarrow T \cup \{(v, w)\}$ ,  $v \leftarrow w$ 
Schließe die Tour:  $T \leftarrow T \cup \{(v, v_0)\}$ 

```

Eulertour

① Hierholzer Algorithmus

Vorgehen:

1. Wähle einen beliebigen Knoten v_0 des Graphen und konstruiere von v_0 ausgehend einen geschlossenen Pfad Z in G , der keine Kante in G zweimal durchläuft. Wir nennen Z einen Zyklus.
2. Wenn Z eine Eulertour ist (also alle Knoten beinhaltet), terminiere. Andernfalls:
3. Lösche nun alle Kanten in Z aus G .
4. An einem Knoten von Z , dessen Grad größer 0 ist, lässt man nun einen Zyklus Z' entstehen, der keine Kante in G zweimal enthält.
5. Füge in Z den zweiten Zyklus Z' ein, indem der Startpunkt von Z' beim ersten Auftreten in Z durch alle Knoten von Z' in der durchlaufenen Reihenfolge ersetzt wird.
6. Fahre bei Schritt 2 fort.

Load-Balancing

② 2-Approximationsalgorithmus

```

List-Scheduling( $m, n, t_1, t_2, \dots, t_n$ ):
  for  $i \leftarrow 1$  bis  $m$ 
     $J_i \leftarrow \emptyset$ 
     $L_i \leftarrow 0$ 
  for  $j \leftarrow 1$  bis  $n$ 
     $i = \operatorname{argmin}_{k=1, \dots, m} L_k$ 
     $J_i \leftarrow J_i \cup \{j\}$ 
     $L_i \leftarrow L_i + t_j$ 
  return  $J_1, \dots, J_m$ 

```

■ Maschine i hat geringste Last

Laufzeit: $O(n \log m)$

m = Anzahl der Maschinen

n = Anzahl der Jobs

Center Selection

② 2-Approximationsalgorithmus (Greedy)

```

Greedy-Center-Selection( $k, n, s_1, s_2, \dots, s_n$ )
 $C = \{s_1\}$ 
wiederhole  $k - 1$  mal
  Wähle einen Standort  $s_i$  mit maximaler  $\operatorname{dist}(s_i, C)$ 
  Füge  $s_i$  zu  $C$  hinzu
return  $C$ 

```

Max-Cut

② 2-Approximations lokale Suche

1-Flip-Nachbarschaft: Gegeben sei eine Partition (A, B) . Verschiebe einen Knoten von A nach B oder von B nach A .

Algorithmus: Ausgehend von einer gültigen Initiallösung ist auf die 1-Flip-Nachbarschaft aufbauend unmittelbar eine einfache lokale Suche möglich.

Allgemein

Branch-and-Bound

① Maximierungsalgorithmus

```

Branch-and-Bound-Max( $I$ ):
 $L \leftarrow -\infty$  oder Wert einer initialen heuristischen Lösung
 $U \leftarrow \infty$  oder obere Schranke für  $I$  aus Dualheuristik
 $\Pi \leftarrow \{(I, L, U)\}$ 
while  $\exists (I', L', U') \in \Pi$ 
    Entferne  $(I', L', U')$  aus  $\Pi$ 
    if  $U' > L$ 
        Partitioniere  $I'$  in Teilinstanzen  $I_1, \dots, I_k$ 
        Berechne zugehörige gültige heuristische Lösungen  $\rightarrow$  untere Schranken  $L_1, \dots, L_k$ 
        Berechne zugehörige lokale obere Schranken  $U_1, \dots, U_k$  mit Dualheuristik
         $\Pi \leftarrow \Pi \cup \{(I_1, L_1, U_1), \dots, (I_k, L_k, U_k)\}$ 
        if  $\max\{L_1, \dots, L_k\} > L$ 
             $L \leftarrow \max\{L_1, \dots, L_k\}$ 
return beste gefundene Lösung mit Wert  $L$ 

```

① Minimierungsalgorithmus

```

Branch-and-Bound-Min( $I$ ):
 $U \leftarrow \infty$  oder Wert einer initialen heuristischen Lösung
 $L \leftarrow -\infty$  oder untere Schranke für  $I$  aus Dualheuristik
 $\Pi \leftarrow \{(I, L, U)\}$ 
while  $\exists (I', L', U') \in \Pi$ 
    Entferne  $(I', L', U')$  aus  $\Pi$ 
    if  $L' < U$ 
        Partitioniere  $I'$  in Teilinstanzen  $I_1, \dots, I_k$ 
        Berechne zugehörige gültige heuristische Lösungen  $\rightarrow$  obere Schranken  $U_1, \dots, U_k$ 
        Berechne zugehörige lokale untere Schranken  $L_1, \dots, L_k$  mit Dualheuristik
         $\Pi \leftarrow \Pi \cup \{(I_1, L_1, U_1), \dots, (I_k, L_k, U_k)\}$ 
        if  $\min\{U_1, \dots, U_k\} < U$ 
             $U \leftarrow \min\{U_1, \dots, U_k\}$ 
return beste gefundene Lösung mit Wert  $U$ 

```

Heuristische Verfahren

Konstruktionsverfahren

Greedy Algorithmus

Greedy-construction-heuristic:

$x \leftarrow$ leere Lösung

while x ist keine vollständige Lösung

$e \leftarrow$ die aktuell nützlichste Erweiterung von x

$x \leftarrow x \oplus e$

Lokale Suche

Prinzip

$x \leftarrow$ Ausgangslösung

while Abbruchkriterium nicht erfüllt

 Wähle $x' \in N(x)$

if x' besser als x

$x \leftarrow x'$;

■ $N(x)$: Nachbarschaft von x

Metaheuristiken

Simulated Annealing

Prinzip

Variablen:

- Z : (Pseudo-)Zufallszahl $\in [0, 1)$
- T : „Temperatur“

```

Simulated-Annealing():
   $t \leftarrow 0$ 
   $T \leftarrow T_{\text{init}}$ 
   $x \leftarrow$  Ausgangslösung
  while Abbruchkriterium nicht erfüllt
    Wähle  $x' \in N(x)$  zufällig
    if  $x'$  besser als  $x$ 
       $x \leftarrow x'$ 
    elseif  $Z < e^{-|f(x') - f(x)|/T}$ 
       $x \leftarrow x'$ 
     $T \leftarrow g(T, t)$ 
     $t \leftarrow t + 1$ 
  
```

■ Metropolis-Kriterium

Tabu-Suche

Prinzip

```

Tabu-Suche():
 $x_{\text{best}} \leftarrow x \leftarrow$  Ausgangslösung
 $TL \leftarrow \{x\}$ 
while Abbruchkriterium nicht erfüllt
     $X' \leftarrow$  Teilmenge von  $N(x)$  unter Berücksichtigung von  $TL$ 
     $x' \leftarrow$  beste Lösung von  $X'$ 
    Füge  $x'$  zu  $TL$  hinzu
    Lösche Elemente aus  $TL$ , welche älter als  $t_L$  Iterationen sind
     $x \leftarrow x'$ 
    if  $x$  besser als  $x_{\text{best}}$ 
         $x_{\text{best}} \leftarrow x$ 

```

Evolutionär

Prinzip

```

Evolutionär():
 $P \leftarrow$  Menge von Ausgangslösungen
Bewerte( $P$ )
while Abbruchkriterium nicht erfüllt
     $Q_s \leftarrow$  Selektion( $P$ )
     $Q_r \leftarrow$  Rekombination( $Q_s$ )
     $P \leftarrow$  Mutation( $Q_r$ )
    Bewerte( $P$ )

```